Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

Lucrarea de laborator Nr. 7

Disciplina: Tehnici de programare

Tema: Git, GitHub și depanare incrementală

A efectuat: st. gr. SI-264
Radu Sologub

A verificat: **Reițman Patricia**, asis. univ.

Chișinău, 2026

Lucrarea de laborator nr. 7

Introducere

Scopul lucrării este înțelegerea operațiilor Git folosite pentru salvarea, publicarea și consultarea istoriei unui proiect. Am folosit proiectul meu existent, SoloDB, o bibliotecă .NET pentru stocarea documentelor într-o bază SQLite. Astfel, exemplele provin din dezvoltarea unui proiect real.

Lucrarea acoperă partea A, „GitHub, pas cu pas”, și partea B, „Unealta stricată, reparată cu git”. Fișele sunt intitulate „Laboratorul 8”, însă în succesiunea lucrărilor disciplinei aceasta este lucrarea nr. 7. În partea A prezint inițializarea, clonarea, salvarea schimbărilor, sincronizarea și citirea istoriei în fluxul SoloDB. În partea B repar programul Python furnizat la laborator și salvez fiecare corecție într-un commit separat în depozitul UTM.

Mediul de lucru a fost Linux, cu Git 2.56.0 și Python 3.14.7. Observațiile din partea A au fost făcute la 20 octombrie 2026, în jurul orei 11:50, ora Chișinăului. Proiectul era în dezvoltare, deci starea ramurii se poate schimba după acest moment. Execuțiile pentru partea B au avut loc în aceeași zi.

1. Contul, configurația și depozitul existent

1.1. Identitatea autorului și ramura principală

Pentru A1–A2 am folosit contul GitHub existent, Unconcurrent. Git este programul care păstrează istoria locală, iar GitHub găzduiește depozitul public. Identitatea înscrisă în commit-uri se verifică separat de autentificarea la GitHub:

```
$ git --version
git version 2.56.0

$ git config --get user.name
Radu

$ git config --get user.email
radu@sologub.md

$ git config --get init.defaultBranch
master
```

Ramura principală este `master`, denumirea deja folosită de SoloDB, în loc de `main` din exemplul fișei. `init.defaultBranch` stabilește numele implicit numai pentru depozitele create ulterior; nu redenumește o ramură existentă. În SoloDB folosesc integrarea prin merge, fără rebase, pentru a păstra originea modificărilor în istorie. Opțiunea `pull.rebase` nu era definită în configurația consultată: interogarea `git config --show-origin --get pull.rebase` nu a afișat text și s-a încheiat cu codul 1. Preferința pentru merge poate fi indicată explicit în comanda pull prin `--no-rebase`.

1.2. Legătura dintre copia locală şi GitHub

Pentru A3–A4 am identificat depozitul şi adresa sa deja configurată. `origin` este numele local al depozitului de la distanţă:

```
$ git remote -v
origin  ssh://git@github.com/Unconcurrent/SoloDB.git (fetch)
origin  ssh://git@github.com/Unconcurrent/SoloDB.git (push)

$ git branch --show-current
master

$ git config --get branch.master.remote
origin

$ git config --get branch.master.merge
refs/heads/master
```

Adresa pentru browser este github.com/Unconcurrent/SoloDB. Adresa Git foloseşte SSH, iar operaţiile autentificate folosesc cheia SSH configurată. Numele şi e-mailul autorului nu ţin locul autentificării. Consultarea API-ului public GitHub a confirmat `private: false` şi ramura implicită `master`.

`git clone` creează o copie locală a unui depozit existent, cu istoria sa şi cu legătura `origin`. Aşa am adus SoloDB pe calculatorul Linux folosit acum ca maşină principală de dezvoltare, după publicarea iniţială a proiectului de pe Windows.

1.3. Originea proiectului pe Windows

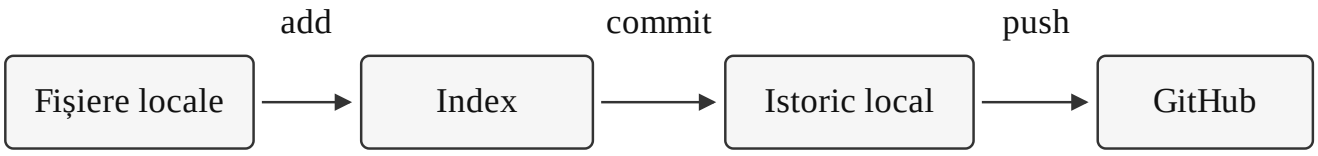
Am creat iniţial depozitul SoloDB pe un calculator Windows, folosind comenzile Git standard şi ramura `master`, apoi l-am publicat pe GitHub. Ulterior l-am clonat pe noul calculator principal de dezvoltare. Calculatorul Windows a rămas o a doua copie de lucru, sincronizată sistematic prin push de pe calculatorul principal şi pull pe Windows.

Acest parcurs corespunde lui A10 pentru pornirea locală şi lui A4 pentru clonarea ulterioară. Rolurile comenzilor sunt: `git init` creează depozitul local, `git add` şi `git commit` salvează fişierele, iar `git remote add origin` asociază adresa de publicare. La prima publicare, forma `git push -u origin master` poate configura şi urmărirea ramurii. În copia actuală, asocierea există deja, după cum arată `branch.master.remote` şi `branch.master.merge`.

2. Starea fişierelor, pregătirea şi commit-ul

2.1. Ce verific înainte de salvare

Pentru A5, fluxul uzual este: inspectez starea şi diferenţele, aleg fişierele care aparţin schimbării, verific ceea ce va intra în commit şi salvez schimbarea cu un mesaj clar. Cele trei niveluri locale sunt fişierele de lucru, indexul şi istoria commit-urilor. Figura 2.1 arată legătura lor cu GitHub.



2.1 - Pregătirea, salvarea locală şi publicarea schimbărilor

`git status --short` foloseşte două coloane pentru starea unui fişier: prima pentru index, a doua pentru fişierul de lucru. `??` indică un fişier neurmărit. Litera `M` în a doua coloană indică o modificare nepregătită, iar în prima coloană o modificare deja pregătită. `git diff` arată diferenţele dintre fişierele de lucru şi index; `git diff --cached` arată diferenţele dintre index şi ultimul commit.

La consultarea din 20 octombrie, ora 11:50:30, rezultatul a fost:

```
$ git status --short --branch
## master...origin/master [ahead 30]
```

Nu erau enumerate fişiere modificate sau neurmărite. Fişierele locale şi indexul erau curate în acel moment, însă ramura avea 30 de commit-uri în plus faţă de referinţa locală `origin/master`. Un director de lucru curat nu înseamnă automat că toate commit-urile sunt publicate.

2.2. Cum se salvează o schimbare în SoloDB

În fluxul proiectului, `git add` pregăteşte căile selectate explicit. Nu publică fişierele şi nu creează singur un commit. După verificarea indexului, `git commit` înregistrează local conţinutul pregătit. Dacă un fişier este editat din nou după `add`, versiunea nouă trebuie pregătită din nou pentru a intra în acelaşi commit. Aceste roluri sunt descrise în documentaţia git-add şi git-commit.

Pentru o schimbare limitată la README, o secvenţă explicativă de salvare este:

```
git diff -- README.md
git add -- README.md
git diff --cached -- README.md
git commit -m "Actualizez documentatia proiectului"
```

Un exemplu existent este commit-ul **Refresh SoloDB documentation and introduction**, din 19 octombrie 2026. Rezultatul consultării sale este:

```
$ git show -1 '--format=%ad | %an | %s' --date=short --stat origin/master
2026-10-19 | Radu | Refresh SoloDB documentation and introduction

 README.md                    | 51 ++++++++++++++++++++++++++----------------
 docs/filesystem-events.md    |  2 ++
 docs/filesystem-references.md |  2 ++
 docs/generated-metadata.md   | 10 +++++++--
 docs/native-sql.md           |  2 ++
 5 files changed, 45 insertions(+), 22 deletions(-)
```

Mesajul descrie actualizarea documentaţiei, iar lista arată fişierele incluse în commit-ul existent.

3. Publicarea şi aducerea schimbărilor

3.1. Push şi dovada unei publicări anterioare

Pentru A6, `git push` trimite commit-urile locale către depozitul de la distanţă. Un commit există local înainte de push şi devine disponibil celeilalte copii după publicare. Fişierele editate, dar neincluse într-un commit, nu sunt trimise prin push. Documentaţia git-push descrie şi asocierea ramurii prin `-u`.

Jurnalul local al referinţei `origin/master` păstrează două actualizări prin push din 19 octombrie. Comanda de consultare executată a fost:

```
git reflog show refs/remotes/origin/master -2 \
  --date=iso-strict '--format=%gD | %gs'
```

Rezultatul a fost:

```
refs/remotes/origin/master@{2026-10-19T20:40:38+03:00} | update by push
refs/remotes/origin/master@{2026-10-19T19:57:46+03:00} | update by push
```

Aceste intrări documentează operaţii anterioare. Separat, consultarea GitHub a confirmat că exemplul de documentaţie şi celelalte trei commit-uri din capitolul 5 sunt publicate. Valoarea `ahead 30` din capitolul 2 compară două referinţe locale; comanda `status` nu interoghează serverul.

3.2. Pull şi actualizarea copiei locale

Pentru A7, exemplul real este actualizarea copiei de pe Windows. După ce public schimbările prin `git push` de pe calculatorul principal, folosesc `git pull` pe Windows pentru a prelua istoria de pe GitHub şi a o integra în copia de acolo. Sincronizarea se repetă în fluxul obişnuit al proiectului. Astfel, acelaşi depozit GitHub leagă cele două calculatoare.

Înaintea unei actualizări verific starea, diferenţele şi existenţa unor schimbări locale care trebuie păstrate. Pentru integrare folosesc merge, fără rebase; forma explicită este `git pull --no-rebase`. Când ramurile au divergat, merge le uneşte printr-un commit cu doi părinţi şi păstrează commit-urile originale, inclusiv legătura lor cu punctul din care au pornit modificările. Dacă este posibil un fast-forward, ramura poate avansa direct, tot fără rescrierea commit-urilor. Aceste comportamente sunt descrise în git-pull şi git-merge.

Rebase ar reaplica modificările pe o altă bază şi ar crea commit-uri noi în locul celor originale. Prefer merge pentru a păstra topologia istoriei şi informaţia despre baza iniţială a lucrului. Traseul push pe Linux, apoi pull pe Windows aplică operaţiile de publicare şi actualizare din fişă celor două calculatoare folosite în dezvoltare.

4. Fişierele excluse prin gitignore

Pentru A8, echivalentul mediului virtual Python din fişă îl constituie fişierele generate de compilarea .NET şi configuraţiile locale ale editorului. SoloDB are deja reguli `.gitignore` pentru directoarele `bin`, `obj`, `.vs`, `.idea` şi `.vscode`, precum şi pentru loguri şi pachete generate.

Am verificat regulile aplicabile unor căi reprezentative:

```
$ git check-ignore -v -- SoloDB/bin/Debug/ SoloDB/obj/
.gitignore:34:[Bb]in/  SoloDB/bin/Debug/
.gitignore:35:[Oo]bj/  SoloDB/obj/
```

Aceste două căi au fost incluse în aceeaşi verificare şi cu exemple pentru `.vs`, `.idea` şi `.vscode`; toate au fost recunoscute de regulile existente. Numerele 34 şi 35 sunt liniile din `.gitignore`, iar expresiile dintre paranteze pătrate admit ambele forme ale primei litere. Interogarea verifică regula chiar dacă o cale de exemplu nu există fizic.

Fişierele proiectului sunt urmărite separat de produsele compilării:

```
$ git ls-files -- README.md SoloDB.sln .gitignore
.gitignore
README.md
SoloDB.sln

$ git ls-files -- 'SoloDB/bin/**' 'SoloDB/obj/**'
```

A doua comandă nu a enumerat fişiere: acele produse ale compilării nu sunt în index. O regulă `.gitignore` se aplică fişierelor neurmărite; nu elimină automat un fişier deja urmărit. De aceea, verific atât regula, cât şi lista fişierelor urmărite. Acest comportament este explicat în documentaţia gitignore.

5. Citirea istoriei şi a diferenţelor

5.1. Istoria compactă

Pentru A9 am afişat data, autorul şi subiectul a patru commit-uri din istoria cunoscută a ramurii publicate:

```
$ git log -4 '--format=%ad | %an | %s' --date=short origin/master
2026-10-19 | Radu | Refresh SoloDB documentation and introduction
2026-10-17 | Radu | Retain array wrapper compatibility checks
2026-10-17 | Radu | Use original assembly scopes for compiled signatures
2026-10-17 | Radu | Preserve original member identity across target profiles
```

Ordinea este de la cel mai recent la cel mai vechi. Consultarea API-ului GitHub pentru `master` a întors aceleaşi patru subiecte, în aceeaşi ordine. Datele din terminal folosesc fusul orar local; API-ul exprimă momentele în UTC, astfel încât un commit din cursul nopţii poate avea altă dată calendaristică.

5.2. O diferenţă reală din README

Am comparat commit-ul de documentaţie cu părintele său. În această observaţie, `origin/master` desemnează acel commit, iar `origin/master~1` părintele său:

```
git diff --unified=1 origin/master~1 origin/master -- README.md
```

La începutul diferenţei apare următorul fragment, reprodus fără linia cu identificatorii obiectelor:

```
diff --git a/README.md b/README.md
--- a/README.md
+++ b/README.md
@@ -1,12 +1,6 @@
-# SoloDB - A Document Database With Full LINQ Support
+# SoloDB
```

Linia cu - aparține versiunii anterioare, iar cea cu + versiunii noi. Contextul @@ indică intervalele din fișierul vechi și din cel nou. Fragmentul arată simplificarea titlului documentației la numele proiectului.

5.3. Retragerea unei schimbări

Fișa folosește `git restore` pentru abandonarea unei editări. Trebuie distinse două operații: `git restore --staged README.md` retrage fișierul din pregătirea pentru commit, păstrând editările locale; `git restore README.md` înlocuiește conținutul de lucru cu versiunea din index și poate pierde editările nepregătite. Implicit, fără `--staged`, sursa restaurării este indexul, nu neapărat ultimul commit. Aceste diferențe sunt precizate în git-restore.

Într-un proiect activ verific mai întâi diferențele fișierului și ce trebuie păstrat.

6. Verificarea părții A și situațiile uzuale

6.1. Corespondența cu cerințele A1–A11

Am verificat identitatea autorului și configurația Git pentru A1–A2, depozitul existent și legătura `origin` pentru A3–A4, starea și un commit real pentru A5, publicările anterioare pentru A6, sincronizarea copiei Windows pentru A7, excluderile pentru A8 și istoria, diferențele și restaurarea pentru A9. Inițializarea pe Windows, publicarea și clonarea ulterioară acoperă cele două direcții de pornire a lucrului prezentate în A4 și A10.

Pentru A11, adresa de predare a acestei variante este **github.com/Unconcurrent/SoloDB**. Istoria publică poate fi consultată la **github.com/Unconcurrent/SoloDB/commits/master**. Depozitul este public, conține README și `.gitignore`, iar cele patru commit-uri prezentate există și pe GitHub.

6.2. Interpretarea situațiilor din A12

`fatal: not a git repository` cere verificarea directorului curent. `Author identity unknown` indică lipsa identității Git, care se configurează înainte de commit. Dacă un fișier apare ca neurmărit și nu intră în commit, verific dacă a fost pregătit prin `add`. Mesajul `nothing to commit, working tree clean` înseamnă că nu există o schimbare locală de salvat; nu confirmă sincronizarea cu GitHub.

Un push respins cu `fetch first` cere consultarea schimbărilor de la distanță și integrarea lor înainte de o nouă publicare. Nu rezolv situația prin suprascrierea forțată a istoriei. O eroare `403` sau `Permission denied` cere verificarea autentificării și a drepturilor pentru adresa folosită. Avertismentul despre conversia LF/CRLF privește sfârșiturile de rând și trebuie interpretat conform regulilor proiectului; nu este, singur, o eroare de commit.

7. Partea B — depanarea programului de inventariere

7.1. Fişierele iniţiale şi metoda de lucru

Am folosit fişierele originale `inventar.py` şi `tinte.json` furnizate pe ELSE. Fişierul JSON conţine şase înregistrări: porturile 22, 80, 3306, 8080, valoarea nenumerică `abc` şi portul 5432. Ultima înregistrare nu are cheia `serviciu`. Programul trebuie să grupeze porturile pe servicii, să ignore portul nevalid şi să afişeze gazdele cu porturi strict mai mari decât pragul ales.

Am salvat versiunea iniţială într-un commit, apoi am rulat programul după fiecare modificare. Am citit ultima linie a erorii şi locul indicat, am corectat o singură problemă şi am făcut un commit separat. La prima rulare, Python a oprit programul înainte de citirea datelor:

```
    def incarca(cale)
                      ^
SyntaxError: expected ':'
```

7.2. Cele zece corecţii în ordinea execuţiei

1. **Declaraţia funcţiei.** `SyntaxError: expected ':'` provenea din lipsa semnului `:` după `def incarca(cale)`. Am completat declaraţia; următoarea rulare a indicat indentarea incorectă din `port_valid`.
2. **Indentarea ramurii except.** Instrucţiunea `return None` trebuia să aparţină blocului `except ValueError`. După corecţie, programul a trecut de analiza sintactică şi a ajuns în `main`.
3. **Numele listei de ţinte.** Expresia `len(tinta)` utiliza un nume inexistent şi producea `NameError`. Am folosit lista încărcată, `tinte`.
4. **Afişarea numărului de ţinte.** Concatenarea unui şir cu rezultatul numeric al lui `len(tinte)` producea `TypeError: can only concatenate str (not "int") to str`. Am folosit `print(f"Tinte citite: {len(tinte)}")`, care a afişat numărul 6.
5. **Serviciul absent.** Accesul `t["serviciu"]` producea `KeyError: 'serviciu'` pentru ultima înregistrare. Am folosit `t.get("serviciu", "necunoscut")`, astfel încât portul 5432 să intre în grupa cerută de fişă.
6. **Rezultatul grupării.** `pe_servicii` construia dicţionarul, dar nu îl întorcea. Apelul ulterior la `grupe.items()` producea `AttributeError: 'NoneType' object has no attribute 'items'`. Am adăugat `return grupe` după buclă.
7. **Tipul pragului.** Argumentul explicit `--prag 1000` era citit ca text, iar comparaţia cu un port întreg producea `TypeError: '>' not supported between instances of 'int' and 'str'`. Am adăugat `type=int` în `add_argument`, păstrând valoarea implicită 1024.
8. **Parcurgerea completă a ţintelor.** Programul pornea, dar afişa `Peste 1000: []`. `return rezultat` se afla în buclă, astfel că filtrarea se încheia după prima ţintă, al cărei port este 22. Am

mutat returnarea după buclă. Verificarea pe primele patru înregistrări valide a întors gazdele `.102` și `.103`; rularea completă a ajuns acum la portul `abc` și a expus următoarea eroare.

9. **Portul nenumeric în filtrare.** Conversia directă `int(t["port"])` producea `ValueError: invalid literal for int() with base 10: 'abc'`. Am reutilizat funcția existentă `port_valid` și am sărit peste rezultatul `None`. Lista peste prag a inclus apoi toate cele trei gazde așteptate.

10. **Păstrarea tuturor porturilor unui serviciu.** Grupa HTTP conținea numai `[8080]`, deoarece `grupe[serviciu] = []` ștergea lista la fiecare înregistrare. Am inițializat lista numai dacă serviciul nu exista încă; rezultatul a devenit `[80, 8080]`.

După corecția a șaptea, lipsa excepțiilor nu însemna că programul era corect. Cele două diferențe față de ieșirea așteptată au indicat greșelile logice. Returnarea prematură ascundea și eroarea de conversie a portului `abc`, de aceea ordinea reală a depanării a inclus o corecție logică înaintea ultimei excepții.

7.3. Gruparea și filtrarea corectate

Am păstrat funcțiile din programul primit. În grupare, cheia absentă primește valoarea implicită, iar o listă existentă este completată:

```
serviciu = t.get("serviciu", "necunoscut")
port = port_valid(t["port"])
if port is None:
    print("sar peste", t["gazda"], ": port invalid", t["port"])
    continue
if serviciu not in grupe:
    grupe[serviciu] = []
grupe[serviciu].append(port)
```

Filtrarea utilizează aceeași conversie și întoarce rezultatul după parcurgerea întregii liste:

```
def peste_prag(tinte, prag):
    """Lista gazdelor cu un port mai mare decat pragul."""
    rezultat = []
    for t in tinte:
        port = port_valid(t["port"])
        if port is None:
            continue
        if port > prag:
            rezultat.append(t["gazda"])
    return rezultat
```

Rolurile excepțiilor și comportamentul lui `dict.get` sunt descrise în documentația Python pentru excepții și dicționare.

7.4. Execuțiile finale și raportul JSON

Rularea cu pragul explicit 1000 a produs exact ieșirea cerută în fișă:

```
$ python inventar.py tinte.json --prag 1000
Tinte citite: 6
sar peste 192.168.56.103 : port invalid abc
ssh: [22]
http: [80, 8080]
mysql: [3306]
necunoscut: [5432]
Peste 1000: ['192.168.56.102', '192.168.56.103', '192.168.56.104']
Raport salvat in raport_inventar.json
```

Am verificat separat și rularea fără opțiunea `--prag`, care folosește 1024:

```
$ python inventar.py tinte.json
Tinte citite: 6
sar peste 192.168.56.103 : port invalid abc
ssh: [22]
http: [80, 8080]
mysql: [3306]
necunoscut: [5432]
Peste 1024: ['192.168.56.102', '192.168.56.103', '192.168.56.104']
Raport salvat in raport_inventar.json
```

Ambele procese s-au încheiat cu codul 0. Listele de gazde sunt identice pentru aceste date, deoarece porturile valide nu au valori între 1000 și 1024. Fișierul generat conține grupele și rezultatul filtrării:

```
{
  "servicii": {
    "ssh": [
      22
    ],
    "http": [
      80,
      8080
    ],
    "mysql": [
      3306
    ],
    "necunoscut": [
      5432
    ]
  },
  "peste_prag": [
    "192.168.56.102",
    "192.168.56.103",
    "192.168.56.104"
  ]
}
```

Am comparat întregul output cu rezultatul așteptat și am verificat separat JSON-ul prin parsare: HTTP conține ambele porturi, portul 5432 apare la `necunoscut`, iar `peste_prag` conține cele trei adrese în ordinea lor din datele inițiale.

7.5. Verificări ale pragului

Pentru pragul 3306, gazda cu portul egal pragului nu trebuie inclusă. Am executat `python inventar.py tinte.json --prag 3306`; fragmentul relevant al rezultatului a fost:

```
Peste 3306: ['192.168.56.103', '192.168.56.104']
```

Un prag nenumeric este respins de `argparse`, înainte de procesarea fişierului, cu codul de ieşire 2:

```
$ python inventar.py tinte.json --prag abc
usage: inventar.py [-h] [--prag PRAG] fisier
inventar.py: error: argument --prag: invalid int value: 'abc'
```

Conversia argumentului prin `type=int` este descrisă în documentația argparse. La final am rulat din nou programul cu pragul 1000 pentru a păstra raportul JSON în forma corespunzătoare exemplului principal.

7.6. Istoria reparaţiilor

Din directorul laboratorului am consultat commit-urile care au modificat exerciţiul, în ordine cronologică:

```
$ git log --reverse --format='%s' -- depanare
Pornire: inventar.py stricat
SyntaxError: adauga doua puncte la incarca
IndentationError: aliniaza ramura except
NameError: foloseste lista tinte la numarare
TypeError: formateaza numarul tintelor ca text
KeyError: foloseste serviciul implicit necunoscut
AttributeError: intoarce grupele calculate
TypeError: converteste pragul la int in argparse
Parcurge toate tintele in filtrarea dupa prag
ValueError: reutilizeaza validarea porturilor
Pastreaza toate porturile aceluiasi serviciu
Raportul uneltei reparate
```

Istoria păstrează versiunea de pornire, cele zece corecţii individuale şi raportul generat. Mesajele descriu schimbările efectiv realizate, iar versiunile intermediare arată cum execuţia a avansat de la erorile de sintaxă la rezultatul complet.

Concluzii

Am urmărit relaţia dintre fişierele de lucru, index, commit-uri şi depozitul GitHub folosind SoloDB. Proiectul a pornit pe Windows şi a ajuns prin clonare pe calculatorul principal, iar copia Windows se actualizează prin pull după publicarea schimbărilor de aici. Am verificat configuraţia şi urmărirea ramurii, am identificat un commit cu cinci fişiere de documentaţie, am citit diferenţa reală din README şi am confirmat regulile care exclud produsele compilării.

Distincţia principală este între salvarea locală şi publicare: un director curat poate avea commit-uri încă nepublicate. `add`, `commit` şi `push` au roluri diferite, iar `pull` integrează schimbările venite din exterior. Folosesc merge, fără rebase, pentru a păstra commit-urile originale şi relaţiile dintre ele. Acelaşi flux leagă dezvoltarea de pe Linux de copia utilizată pe Windows.

În partea B am corectat cele zece probleme ale programului primit, cu un commit după fiecare reparaţie. Am verificat cele două rulări cerute, conţinutul JSON, comparaţia strictă cu pragul şi respingerea unui prag nenumeric. Depanarea a arătat că un program care se încheie fără excepţii poate produce încă rezultate greşite şi că o returnare prea devreme poate ascunde o altă eroare.

Bibliografie

1. UTM, FCIM. *Laboratorul 8, Lucrarea A. GitHub, pas cu pas* și *Lucrarea B. Unealta stricată, reparată cu git.* Fișele disciplinei Tehnici de programare și fișierele originale furnizate pe ELSE, utilizate pentru lucrarea nr. 7.
2. Radu Sologub. SoloDB — depozitul public și istoricul proiectului.
3. Git. gittutorial — introducere în operațiile de bază și git-clone.
4. Git. git-add și git-commit.
5. Git. git-push, git-pull, git-merge și git-rebase.
6. Git. gitignore și git-restore.
7. Python Software Foundation. argparse, Built-in Exceptions și dict.get.

Sursele online au fost consultate la 20 octombrie 2026.